

Практическое занятие 33: Подготовка проекта к деплою

Цель занятия:

- Подготовить ваш Django-проект к развертыванию на сервере.
 - Настроить необходимые параметры и конфигурации для работы в продакшн-среде.
 - Создать файл зависимостей `requirements.txt`.
 - Настроить статические файлы и медиа-файлы.
 - Обеспечить безопасность приложения путем настройки секретных ключей и параметров отладки.
-

Задачи занятия

1. Создание файла зависимостей `requirements.txt`
 2. Настройка настроек проекта для продакшн-среды
 3. Настройка статических файлов
 4. Настройка файла WSGI/ASGI
 5. Проверка проекта перед деплоем
-

Задача 1: Создание файла зависимостей `requirements.txt`

1.1. Что такое `requirements.txt` ?

- `requirements.txt` — это текстовый файл, содержащий список всех пакетов Python, необходимых для работы вашего проекта, с указанием их версий.
- При деплое на сервер вы можете использовать этот файл для установки всех зависимостей проекта с помощью команды `pip install -r requirements.txt`.

1.2. Создание файла `requirements.txt`

Шаги:

1. Активируйте виртуальное окружение вашего проекта.

```
# В зависимости от того, какое виртуальное окружение вы используете
source venv/bin/activate # Для Unix/macOS
venv\Scripts\activate   # Для Windows
```

2. Сгенерируйте файл `requirements.txt`.

В корневой директории вашего проекта выполните команду:

```
pip freeze > requirements.txt
```

3. Проверьте содержимое файла `requirements.txt`.

Откройте файл и убедитесь, что в нем перечислены все необходимые пакеты, например:

```
Django==4.2
django-rest-framework==3.14.0
gunicorn==20.1.0
psycopg2-binary==2.9.3
# и другие пакеты
```

Примечание: Убедитесь, что версии пакетов указаны корректно и соответствуют тем, которые вы использовали при разработке.

Задача 2: Настройка настроек проекта для продакшн-среды

2.1. Разделение настроек для разработки и продакшна

Шаги:

1. Создайте файл `settings.py` для продакшн-среды.

В директории вашего приложения (например, `your_project/your_project/`) создайте файл `production_settings.py`.

2. Скопируйте содержимое вашего основного `settings.py` в `production_settings.py`.

```
cp your_project/settings.py your_project/production_settings.py
```

3. В основном `settings.py` настройте параметры для разработки.

4. В `production_settings.py` настройте параметры для продакшна.

Основные отличия:

- `DEBUG = False`

В `production_settings.py` установите:

```
DEBUG = False
```

- **Настройка** `ALLOWED_HOSTS`

Укажите доменные имена и IP-адреса, на которых будет доступен ваш сайт:

```
ALLOWED_HOSTS = ['yourdomain.com', 'www.yourdomain.com',  
'IP_ADDRESS']
```

- **Настройка секретного ключа**

Никогда не храните секретный ключ в публичных репозиториях!

В `production_settings.py` задайте:

```
SECRET_KEY = os.environ.get('DJANGO_SECRET_KEY', 'your-production-  
secret-key')
```

Примечание: Лучше всего хранить секретный ключ в переменных окружения.

- **Настройка базы данных**

В продакшн-среде обычно используется PostgreSQL или другая СУБД.

```
DATABASES = {  
    'default': {  
        'ENGINE': 'django.db.backends.postgresql_psycopg2',  
        'NAME': 'your_db_name',  
        'USER': 'your_db_user',  
        'PASSWORD': 'your_db_password',  
        'HOST': 'your_db_host',  
        'PORT': 'your_db_port',  
    }  
}
```

- **Безопасность**

Добавьте или настройте следующие параметры:

```
SECURE_SSL_REDIRECT = True
SESSION_COOKIE_SECURE = True
CSRF_COOKIE_SECURE = True
X_FRAME_OPTIONS = 'DENY'
```

2.2. Использование переменных окружения

Шаги:

1. Установите пакет `python-decouple` для работы с переменными окружения:

```
pip install python-decouple
```

2. Добавьте его в `requirements.txt`:

```
echo 'python-decouple' >> requirements.txt
```

3. Обновите настройки в `production_settings.py`:

```
from decouple import config

SECRET_KEY = config('DJANGO_SECRET_KEY')
DEBUG = config('DEBUG', default=False, cast=bool)
ALLOWED_HOSTS = config('ALLOWED_HOSTS', cast=lambda v: [s.strip() for s
in v.split(',')])
```

4. Создайте файл `.env` для хранения переменных окружения (на сервере).

Пример содержимого `.env`:

```
DJANGO_SECRET_KEY=your-production-secret-key
DEBUG=False
ALLOWED_HOSTS=yourdomain.com,www.yourdomain.com,IP_ADDRESS
```

Примечание: Никогда не добавляйте файл `.env` в систему контроля версий (например, Git). Добавьте его в `.gitignore`.

Задача 3: Настройка статических файлов

3.1. Настройка `STATIC_ROOT`

В продакшн-среде статические файлы должны быть собраны в одну директорию, чтобы веб-сервер мог их обслуживать.

Шаги:

1. В `production_settings.py` установите путь для `STATIC_ROOT`:

```
STATIC_ROOT = os.path.join(BASE_DIR, 'staticfiles')
```

2. Убедитесь, что `STATIC_URL` настроен корректно:

```
STATIC_URL = '/static/'
```

3.2. Сборка статических файлов

Шаги:

1. Выполните команду для сбора статических файлов:

```
python manage.py collectstatic
```

Примечание: Эта команда собирает все статические файлы из ваших приложений и пакетов в директорию `STATIC_ROOT`.

3.3. Настройка медиа-файлов (если используется)

Шаги:

1. В `production_settings.py` настройте `MEDIA_ROOT` и `MEDIA_URL`:

```
MEDIA_ROOT = os.path.join(BASE_DIR, 'media')
MEDIA_URL = '/media/'
```

2. **Убедитесь, что веб-сервер настроен на обслуживание медиа-файлов из `MEDIA_ROOT`.

Задача 4: Настройка файла WSGI/ASGI

4.1. Проверка файла `wsgi.py`

Шаги:

1. Откройте файл `wsgi.py` в директории вашего проекта (`your_project/your_project/wsgi.py`).
2. Убедитесь, что приложение использует настройки продакшн:

```
import os

from django.core.wsgi import get_wsgi_application

os.environ.setdefault('DJANGO_SETTINGS_MODULE',
'your_project.production_settings')

application = get_wsgi_application()
```

Примечание: Замените `'your_project.production_settings'` на путь к вашим продакшн-настройкам.

4.2. (Опционально) Настройка файла `asgi.py` для асинхронных серверов

Если вы планируете использовать ASGI-сервер (например, Daphne или Uvicorn), настройте файл `asgi.py` аналогично:

```
import os

from django.core.asgi import get_asgi_application

os.environ.setdefault('DJANGO_SETTINGS_MODULE',
'your_project.production_settings')

application = get_asgi_application()
```

Задача 5: Проверка проекта перед деплоем

5.1. Запуск тестового сервера с продакшн-настройками

Шаги:

1. Убедитесь, что у вас есть база данных, соответствующая настройкам в `production_settings.py`.
2. Примените миграции:

```
python manage.py migrate --settings=your_project.production_settings
```

3. Запустите сервер с использованием продакшн-настроек:

```
python manage.py runserver --settings=your_project.production_settings
```

4. Проверьте, что приложение работает без ошибок.

5.2. Запуск тестов (если имеются)

Шаги:

1. Выполните тесты с продакшн-настройками:

```
python manage.py test --settings=your_project.production_settings
```

2. Убедитесь, что все тесты проходят успешно.

Дополнительные рекомендации

- **Проверьте наличие уязвимостей:**
 - Используйте команды или инструменты для проверки безопасности вашего проекта, например, `bandit` или `safety`.
- **Настройте логирование:**
 - В продакшн-среде важно иметь корректное логирование для отслеживания ошибок и мониторинга.

```
# В production_settings.py

LOGGING = {
    'version': 1,
    'disable_existing_loggers': False,
    'handlers': {
        'file': {
            'level': 'ERROR',
            'class': 'logging.FileHandler',
            'filename': os.path.join(BASE_DIR, 'logs', 'django.log'),
        },
    },
    'loggers': {
        'django': {
            'handlers': ['file'],
            'level': 'ERROR',
            'propagate': True,
        },
    },
}
```

```
} ,  
}
```

- **Настройте кэширование и производительность:**
 - Рассмотрите возможность использования кэша (Redis, Memcached) для улучшения производительности.
 - Используйте Gzip-сжатие и другие оптимизации на уровне веб-сервера.
-

Заключение

Поздравляем! Вы успешно:

- Создали файл `requirements.txt` с зависимостями вашего проекта.
 - Настроили отдельные файлы настроек для разработки и продакшн-среды.
 - Настроили обработку статических и медиа-файлов для продакшн-среды.
 - Обновили файл `wsgi.py` для использования продакшн-настроек.
 - Провели предварительную проверку проекта перед деплоем.
-

Следующим шагом будет непосредственный деплой вашего проекта на сервер, что будет рассмотрено в Практическом занятии 32.

Если у вас возникнут дополнительные вопросы или потребуется помощь с каким-либо шагом, пожалуйста, дайте мне знать.

Успехов в разработке и подготовке к деплою!

Вопросы для обсуждения

- **Какую роль играет файл `requirements.txt` при деплое Django-проекта?**
- Файл `requirements.txt` содержит список всех зависимостей проекта с указанием их версий. При деплое на сервере его используют для установки всех необходимых пакетов командой `pip install -r requirements.txt`. Это обеспечивает консистентность среды и гарантирует, что проект будет работать с теми же версиями библиотек, что и во время разработки.
- **Почему важно разделять настройки для разработки и продакшн-среды в Django?**
- Разделение настроек позволяет оптимизировать конфигурацию под конкретную среду. В разработке `DEBUG` обычно включен для удобства отладки, тогда как в

продакшн-среде его необходимо отключить для безопасности. Также различаются параметры базы данных, разрешенные хосты и другие конфигурации, что помогает обеспечить безопасность и производительность приложения.

- **Каковы основные шаги настройки статических файлов для продакшн-среды в Django?**
 - Основные шаги включают установку `STATIC_ROOT` в настройках проекта, что указывает Django, куда будут собираться все статические файлы. Затем выполняется команда `python manage.py collectstatic` для сбора всех статических файлов в указанную директорию. После этого веб-сервер (например, Nginx) настраивается для обслуживания этих статических файлов, обеспечивая быстрый доступ и оптимизацию производительности приложения.
-

Полезные ресурсы

- [Официальная документация Django по развертыванию](#)
- [Руководство по настройке продакшн-настроек](#)
- [Python Decouple](#)
- [Настройка статических файлов в Django](#)